

```
import { useState, useEffect } from "react";
```

```
// ——— CONFIG
```

```
const SUPER_ADMIN_PLACA = "MX-06";  
const SUPER_ADMIN_PASS = "kracccea2026"; // cámbiala tú mismo aquí
```

```
// ——— HELPERS
```

```
const store = {  
  get: (k) => { try { return JSON.parse(localStorage.getItem(k)); } catch { return null; } },  
  set: (k, v) => localStorage.setItem(k, JSON.stringify(v)),  
};
```

```
const initStore = () => {  
  if (!store.get("ssc_elements")) store.set("ssc_elements", []);  
  if (!store.get("ssc_admins")) store.set("ssc_admins", []);  
  if (!store.get("ssc_reports")) store.set("ssc_reports", []);  
};
```

```
const now = () => new Date().toLocaleString("es-MX", { timeZone: "America/Mexico_City" });
```

```
// ——— ICONS
```

```
const Icon = {  
  shield: (  
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"  
style={{width:20,height:20}}>  
      <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"/>  
    </svg>  
  ),  
  user: (  
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"  
style={{width:20,height:20}}>  
      <path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"/><circle cx="12" cy="7" r="4"/>  
    </svg>  
  ),  
  star: (  
    <svg viewBox="0 0 24 24" fill="currentColor" style={{width:16,height:16}}>  
      <polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18 21.02 12 17.77 5.82 21.02 7
```

```

14.14 2 9.27 8.91 8.26 12 2"/>
  </svg>
),
trash: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:16,height:16}}>
    <polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3
0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"/>
  </svg>
),
plus: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:18,height:18}}>
    <line x1="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/>
  </svg>
),
logout: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:18,height:18}}>
    <path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><polyline points="16 17 21 12 16
7"/><line x1="21" y1="12" x2="9" y2="12"/>
  </svg>
),
edit: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:16,height:16}}>
    <path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7"/><path d="M18.5
2.5a2.121 2.121 0 0 1 3 3L12 15l-4 1 1-4 9.5-9.5z"/>
  </svg>
),
check: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5"
style={{width:16,height:16}}>
    <polyline points="20 6 9 17 4 12"/>
  </svg>
),
file: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:20,height:20}}>
    <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline
points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8"
y2="17"/><polyline points="10 9 9 9 8 9"/>
  </svg>

```

```

),
users: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:20,height:20}}>
    <path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path
d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0 7.75"/>
  </svg>
),
badge: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:20,height:20}}>
    <rect x="2" y="7" width="20" height="14" rx="2"/><path d="M16 21V5a2 2 0 0 0-2-2h-4a2 2
0 0 0-2 2v16"/>
  </svg>
),
warning: (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
style={{width:16,height:16}}>
    <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0
0-3.42 0z"/><line x1="12" y1="9" x2="12" y2="13"/><line x1="12" y1="17" x2="12.01" y2="17"/>
  </svg>
),
};

```

// ——— STYLES

```

const CSS = `
  @import
url('https://fonts.googleapis.com/css2?family=Rajdhani:wght@400;500;600;700&family=Share+
Tech+Mono&family=Barlow:wght@300;400;500;600&display=swap');

```

```

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

```

```

:root {
  --navy: #0a0e1a;
  --panel: #0f1525;
  --card: #141c30;
  --border: #1e2d4a;
  --accent: #1e6fff;
  --green: #00c47a;
  --red: #ff3b55;
  --gold: #f5c842;
}

```

```

--text: #c8d8f0;
--muted: #5a7099;
--white: #e8f0ff;
}

body { background: var(--navy); color: var(--text); font-family: 'Barlow', sans-serif; min-height: 100vh; }

/* scanline overlay */
body::before {
  content: "";
  position: fixed; inset: 0; pointer-events: none; z-index: 9999;
  background: repeating-linear-gradient(0deg, transparent, transparent 2px, rgba(0,0,0,.04) 2px, rgba(0,0,0,.04) 4px);
}

h1,h2,h3,h4 { font-family: 'Rajdhani', sans-serif; letter-spacing: .05em; }

/* — Layout — */
.app { display: flex; flex-direction: column; min-height: 100vh; }

/* — Topbar — */
.topbar {
  display: flex; align-items: center; justify-content: space-between;
  padding: 0 28px; height: 64px;
  background: var(--panel);
  border-bottom: 1px solid var(--border);
  position: sticky; top: 0; z-index: 100;
}
.topbar-brand { display: flex; align-items: center; gap: 12px; }
.topbar-logo {
  width: 40px; height: 40px; border-radius: 50%;
  background: linear-gradient(135deg, var(--accent), #0040cc);
  display: flex; align-items: center; justify-content: center;
  font-family: 'Rajdhani', sans-serif; font-weight: 700; font-size: 13px;
  color: #fff; letter-spacing: .05em;
  box-shadow: 0 0 16px rgba(30,111,255,.4);
}
.topbar-title { font-family: 'Rajdhani', sans-serif; font-size: 18px; font-weight: 700; color: var(--white); }
.topbar-sub { font-size: 11px; color: var(--muted); letter-spacing: .1em; text-transform: uppercase; }
.topbar-right { display: flex; align-items: center; gap: 16px; }

```

```

.placa-badge {
  font-family: 'Share Tech Mono', monospace; font-size: 12px;
  padding: 4px 10px; border-radius: 4px;
  background: rgba(30,111,255,.15); border: 1px solid rgba(30,111,255,.3);
  color: var(--accent);
}

.role-badge {
  font-size: 11px; padding: 3px 8px; border-radius: 3px;
  font-weight: 600; letter-spacing: .08em; text-transform: uppercase;
}

.role-superadmin { background: rgba(245,200,66,.12); border: 1px solid rgba(245,200,66,.3);
color: var(--gold); }
.role-admin { background: rgba(0,196,122,.12); border: 1px solid rgba(0,196,122,.3); color:
var(--green); }
.role-element { background: rgba(30,111,255,.12); border: 1px solid rgba(30,111,255,.3);
color: var(--accent); }

/* — Buttons — */
.btn {
  display: inline-flex; align-items: center; gap: 7px;
  padding: 8px 16px; border-radius: 5px; border: none; cursor: pointer;
  font-family: 'Barlow', sans-serif; font-size: 13px; font-weight: 500;
  transition: all .18s; white-space: nowrap;
}
.btn:disabled { opacity: .4; cursor: not-allowed; }
.btn-primary { background: var(--accent); color: #fff; }
.btn-primary:hover:not(:disabled) { background: #3a7fff; box-shadow: 0 0 18px
rgba(30,111,255,.4); }
.btn-green { background: var(--green); color: #001a0e; }
.btn-green:hover:not(:disabled) { box-shadow: 0 0 18px rgba(0,196,122,.35); }
.btn-red { background: rgba(255,59,85,.15); border: 1px solid rgba(255,59,85,.3); color:
var(--red); }
.btn-red:hover:not(:disabled) { background: rgba(255,59,85,.25); }
.btn-ghost { background: rgba(255,255,255,.04); border: 1px solid var(--border); color:
var(--text); }
.btn-ghost:hover:not(:disabled) { background: rgba(255,255,255,.08); }
.btn-gold { background: rgba(245,200,66,.15); border: 1px solid rgba(245,200,66,.3); color:
var(--gold); }
.btn-gold:hover:not(:disabled) { background: rgba(245,200,66,.25); }
.btn-sm { padding: 5px 11px; font-size: 12px; }

/* — Main content — */
.main { flex: 1; padding: 32px 32px; max-width: 1200px; margin: 0 auto; width: 100%; }

```

```

/* — Login — */
.login-wrap {
  min-height: 100vh; display: flex; align-items: center; justify-content: center;
  background: var(--navy);
  background-image: radial-gradient(ellipse 60% 60% at 50% 0%, rgba(30,111,255,.12) 0%,
transparent 70%);
}
.login-box {
  width: 380px; padding: 44px 36px;
  background: var(--panel); border: 1px solid var(--border); border-radius: 12px;
  box-shadow: 0 24px 64px rgba(0,0,0,.5);
}
.login-logo-wrap { text-align: center; margin-bottom: 28px; }
.login-logo {
  width: 72px; height: 72px; border-radius: 50%; margin: 0 auto 12px;
  background: linear-gradient(135deg, var(--accent), #003ab0);
  display: flex; align-items: center; justify-content: center;
  box-shadow: 0 0 32px rgba(30,111,255,.5);
  font-family: 'Rajdhani', sans-serif; font-weight: 700; font-size: 22px; color: #fff;
}
.login-title { font-size: 24px; font-weight: 700; color: var(--white); text-align: center; }
.login-sub { font-size: 12px; color: var(--muted); text-align: center; letter-spacing: .12em;
text-transform: uppercase; margin-top: 4px; }

.field { margin-bottom: 18px; }
.field label { display: block; font-size: 11px; font-weight: 600; letter-spacing: .1em;
text-transform: uppercase; color: var(--muted); margin-bottom: 6px; }
.field input, .field select, .field textarea {
  width: 100%; padding: 10px 14px; border-radius: 6px;
  background: rgba(255,255,255,.04); border: 1px solid var(--border);
  color: var(--white); font-family: 'Barlow', sans-serif; font-size: 14px;
  outline: none; transition: border .15s;
}
.field input:focus, .field select:focus, .field textarea:focus { border-color: var(--accent); }
.field textarea { resize: vertical; min-height: 80px; }
.field select option { background: var(--panel); }
.field input::placeholder { color: var(--muted); }

.error-msg { background: rgba(255,59,85,.1); border: 1px solid rgba(255,59,85,.25); color:
var(--red); border-radius: 5px; padding: 9px 13px; font-size: 13px; margin-bottom: 16px; display:
flex; align-items: center; gap: 7px; }
.success-msg { background: rgba(0,196,122,.1); border: 1px solid rgba(0,196,122,.25); color:

```

```

var(--green); border-radius: 5px; padding: 9px 13px; font-size: 13px; margin-bottom: 16px; }

.divider { height: 1px; background: var(--border); margin: 20px 0; }
.link-btn { background: none; border: none; cursor: pointer; color: var(--accent); font-size: 13px;
text-decoration: underline; font-family: 'Barlow', sans-serif; }

/* — Cards grid — */
.cards-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap:
20px; margin-top: 24px; }

.menu-card {
  background: var(--card); border: 1px solid var(--border); border-radius: 10px;
  padding: 36px 28px; cursor: pointer; transition: all .2s;
  display: flex; flex-direction: column; align-items: center; gap: 14px; text-align: center;
}
.menu-card:hover { border-color: var(--accent); transform: translateY(-2px); box-shadow: 0
12px 32px rgba(0,0,0,.3); }
.menu-card-icon { font-size: 40px; }
.menu-card-title { font-size: 22px; font-weight: 700; color: var(--white); }
.menu-card-desc { font-size: 13px; color: var(--muted); line-height: 1.5; }

/* — Section header — */
.section-hdr { display: flex; align-items: center; justify-content: space-between; margin-bottom:
20px; flex-wrap: gap; gap: 12px; }
.section-hdr h2 { font-size: 22px; color: var(--white); display: flex; align-items: center; gap:
10px; }

/* — Table — */
.tbl-wrap { background: var(--card); border: 1px solid var(--border); border-radius: 10px;
overflow: hidden; }
table { width: 100%; border-collapse: collapse; }
th { background: rgba(255,255,255,.03); padding: 11px 16px; text-align: left; font-size: 11px;
font-weight: 600; letter-spacing: .1em; text-transform: uppercase; color: var(--muted);
border-bottom: 1px solid var(--border); }
td { padding: 12px 16px; font-size: 13px; border-bottom: 1px solid rgba(30,45,74,.6);
vertical-align: middle; }
tr:last-child td { border-bottom: none; }
tr:hover td { background: rgba(255,255,255,.015); }
.mono { font-family: 'Share Tech Mono', monospace; font-size: 13px; }

/* — Modal — */
.modal-overlay {
  position: fixed; inset: 0; background: rgba(0,0,0,.7); backdrop-filter: blur(4px);

```

```

    display: flex; align-items: center; justify-content: center; z-index: 500; padding: 20px;
}
.modal {
    background: var(--panel); border: 1px solid var(--border); border-radius: 12px;
    width: 100%; max-width: 520px; max-height: 90vh; overflow-y: auto;
    padding: 32px; box-shadow: 0 32px 80px rgba(0,0,0,.6);
}
.modal h3 { font-size: 20px; color: var(--white); margin-bottom: 20px; display: flex; align-items:
center; gap: 10px; }

/* — Profile card — */
.profile-card {
    background: var(--card); border: 1px solid var(--border); border-radius: 10px; padding: 28px;
}
.profile-header { display: flex; align-items: center; gap: 20px; margin-bottom: 24px; }
.profile-avatar {
    width: 72px; height: 72px; border-radius: 50%;
    background: linear-gradient(135deg, var(--accent), #003ab0);
    display: flex; align-items: center; justify-content: center;
    font-family: 'Rajdhani', sans-serif; font-weight: 700; font-size: 24px; color: #fff;
    box-shadow: 0 0 24px rgba(30,111,255,.35); flex-shrink: 0;
}
.profile-name { font-size: 22px; font-weight: 700; color: var(--white); }
.profile-placa { font-family: 'Share Tech Mono', monospace; font-size: 14px; color: var(--accent); }
}
.profile-rank { font-size: 13px; color: var(--muted); margin-top: 3px; }

.info-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
.info-item label { font-size: 11px; font-weight: 600; letter-spacing: .1em; text-transform:
uppercase; color: var(--muted); display: block; margin-bottom: 4px; }
.info-item span { font-size: 14px; color: var(--white); }

/* — Record entries — */
.record-list { display: flex; flex-direction: column; gap: 10px; margin-top: 12px; }
.record-item {
    background: rgba(255,255,255,.03); border: 1px solid var(--border); border-radius: 7px;
    padding: 12px 16px; display: flex; justify-content: space-between; align-items: flex-start; gap:
12px;
}
.record-item.sancion { border-left: 3px solid var(--red); }
.record-item.elogia { border-left: 3px solid var(--green); }
.record-item.nota { border-left: 3px solid var(--accent); }
.record-meta { font-size: 11px; color: var(--muted); margin-top: 4px; }

```



```

.tag {
  font-size: 10px; font-weight: 700; letter-spacing: .08em; text-transform: uppercase;
  padding: 2px 8px; border-radius: 3px;
}
.tag-sancion { background: rgba(255,59,85,.15); color: var(--red); border: 1px solid
rgba(255,59,85,.25); }
.tag-elogio { background: rgba(0,196,122,.15); color: var(--green); border: 1px solid
rgba(0,196,122,.25); }
.tag-nota { background: rgba(30,111,255,.15);color: var(--accent); border: 1px solid
rgba(30,111,255,.25); }

/* — Status chips — */
.chip { display: inline-flex; align-items: center; gap: 5px; font-size: 11px; font-weight: 600;
padding: 3px 9px; border-radius: 20px; }
.chip-active { background: rgba(0,196,122,.12); color: var(--green); border: 1px solid
rgba(0,196,122,.25); }
.chip-baja { background: rgba(255,59,85,.12); color: var(--red); border: 1px solid
rgba(255,59,85,.25); }
.chip-suspend { background: rgba(245,200,66,.12); color: var(--gold); border: 1px solid
rgba(245,200,66,.25); }

/* — Tabs — */
.tabs { display: flex; gap: 4px; background: rgba(255,255,255,.04); border-radius: 7px; padding:
4px; margin-bottom: 20px; }
.tab-btn { flex: 1; padding: 8px; border-radius: 5px; border: none; cursor: pointer; font-family:
'Barlow', sans-serif; font-size: 13px; font-weight: 500; transition: all .15s; background: none;
color: var(--muted); }
.tab-btn.active { background: var(--accent); color: #fff; }

/* — Topbar back btn — */
.back-btn { background: none; border: none; cursor: pointer; color: var(--muted); font-size:
13px; display: flex; align-items: center; gap: 6px; font-family: 'Barlow', sans-serif; transition: color
.15s; }
.back-btn:hover { color: var(--white); }

/* — Stats row — */
.stats-row { display: grid; grid-template-columns: repeat(auto-fill, minmax(180px, 1fr)); gap:
14px; margin-bottom: 28px; }
.stat-card { background: var(--card); border: 1px solid var(--border); border-radius: 8px;
padding: 18px 20px; }
.stat-value { font-family: 'Rajdhani', sans-serif; font-size: 32px; font-weight: 700; color:
var(--white); }
.stat-label { font-size: 11px; color: var(--muted); text-transform: uppercase; letter-spacing: .1em;

```

```
margin-top: 2px; }
```

```
/* — empty state — */
```

```
.empty { text-align: center; padding: 48px 20px; color: var(--muted); font-size: 14px; }
```

```
/* — toast — */
```

```
.toast {
```

```
  position: fixed; bottom: 28px; right: 28px; z-index: 999;
```

```
  background: var(--panel); border: 1px solid var(--border); border-radius: 8px;
```

```
  padding: 12px 18px; font-size: 13px; color: var(--white);
```

```
  box-shadow: 0 8px 32px rgba(0,0,0,.4);
```

```
  animation: slideIn .25s ease;
```

```
}
```

```
@keyframes slideIn { from { transform: translateY(20px); opacity:0; } to { transform: translateY(0); opacity:1; } }
```

```
/* — search box — */
```

```
.search-box { position: relative; }
```

```
.search-box input { padding-left: 36px; }
```

```
.search-box::before {
```

```
  content: '🔍'; position: absolute; left: 10px; top: 50%; transform: translateY(-50%);
```

```
  font-size: 13px; pointer-events: none;
```

```
}
```

```
@media (max-width: 640px) {
```

```
  .main { padding: 16px; }
```

```
  .topbar { padding: 0 14px; }
```

```
  .info-grid { grid-template-columns: 1fr; }
```

```
}
```

```
`;
```

```
// — COMPONENTS
```

```
function Toast({ msg, onClose }) {  
  useEffect(() => {  
    const t = setTimeout(onClose, 3000);  
    return () => clearTimeout(t);  
  }, []);  
  return <div className="toast">{msg}</div>;  
}
```

```

function Modal({ title, onClose, children }) {
  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget && onClose()}>
      <div className="modal">
        <h3>{title}</h3>
        {children}
        <div style={{ marginTop: 12 }}>
          <button className="btn btn-ghost btn-sm" onClick={onClose}>Cancelar</button>
        </div>
      </div>
    </div>
  );
}

```

// — APP

```

export default function App() {
  const [session, setSession] = useState(null); // { placa, role, nombre }
  const [view, setView] = useState("home");
  const [toast, setToast] = useState(null);
  const [elements, setElements] = useState([]);
  const [admins, setAdmins] = useState([]);
  const [reports, setReports] = useState([]);

  useEffect(() => {
    initStore();
    setElements(store.get("ssc_elements") || []);
    setAdmins(store.get("ssc_admins") || []);
    setReports(store.get("ssc_reports") || []);
  }, []);

  const saveElements = (data) => { setElements(data); store.set("ssc_elements", data); };
  const saveAdmins = (data) => { setAdmins(data); store.set("ssc_admins", data); };
  const saveReports = (data) => { setReports(data); store.set("ssc_reports", data); };

  const showToast = (msg) => setToast(msg);

  const handleLogin = ({ placa, password }) => {
    const p = placa.trim().toUpperCase();
    // Super admin
    if (p === SUPER_ADMIN_PLACA && password === SUPER_ADMIN_PASS) {

```

```

    setSession({ placa: p, role: "superadmin", nombre: "kracccea" });
    setView("home");
    return true;
  }
  // Check admin list
  const isAdmin = admins.find(a => a.placa === p && a.password === password);
  if (isAdmin) {
    setSession({ placa: p, role: "admin", nombre: isAdmin.nombre });
    setView("home");
    return true;
  }
  // Check element
  const el = elements.find(e => e.placa === p && e.password === password);
  if (el) {
    setSession({ placa: p, role: "element", nombre: el.nombre });
    setView("home");
    return true;
  }
  return false;
};

const logout = () => { setSession(null); setView("home"); };

if (!session) {
  return (
    <>
    <style>{CSS}</style>
    <LoginView onLogin={handleLogin} onRegister={(data) => {
      if (elements.find(e => e.placa === data.placa)) return "Esa placa ya está registrada.";
      const newEl = { ...data, records: [], createdAt: now(), createdBy: "Autoregistro", status:
"activo" };
      saveElements([...elements, newEl]);
      return null;
    }} />
    </>
  );
}

return (
  <>
  <style>{CSS}</style>
  <div className="app">
    <Topbar session={session} onLogout={logout} onBack={view !== "home" ? () =>

```

```

setView("home") : null} />
<div className="main">
  {view === "home" && (
    <HomeView session={session} onNav={setView} elements={elements} admins={admins}
  />
  )}
  {view === "elementos" && (
    <ElementosView
      session={session}
      elements={elements}
      admins={admins}
      onSave={saveElements}
      toast={showToast}
    />
  )}
  {view === "admins" && session.role === "superadmin" && (
    <AdminsView
      admins={admins}
      elements={elements}
      onSave={saveAdmins}
      toast={showToast}
    />
  )}
  {view === "miperfil" && (
    <MiPerfilView
      session={session}
      elements={elements}
      onSave={saveElements}
      toast={showToast}
    />
  )}
  {view === "reportes" && (
    <ReportesView
      session={session}
      elements={elements}
      reports={reports}
      onSave={saveReports}
      toast={showToast}
    />
  )}
</div>
{toast && <Toast msg={toast} onClose={() => setToast(null)} />}
</div>

```

```

    </>
  );
}

// ——— TOPBAR


---



function Topbar({ session, onLogout, onBack }) {
  const roleLabel = { superadmin: "Super Admin", admin: "Administrador", element: "Elemento" };
  const roleCls = { superadmin: "role-superadmin", admin: "role-admin", element: "role-element" };
  return (
    <div className="topbar">
      <div className="topbar-brand">
        {onBack && (
          <button className="back-btn" onClick={onBack}>
            ← Volver
          </button>
        )}
      <div className="topbar-logo">SSC</div>
      <div>
        <div className="topbar-title">SSC CDMX</div>
        <div className="topbar-sub">Sistema de Gestión Interna</div>
      </div>
      <div className="topbar-right">
        <span className="placa-badge">{session.placa}</span>
        <span className={`role-badge ${roleCls[session.role]}`}>{roleLabel[session.role]}</span>
        <button className="btn btn-ghost btn-sm" onClick={onLogout}>{Icon.logout}
          Salir</button>
      </div>
    </div>
  );
}

// ——— LOGIN


---



function LoginView({ onLogin, onRegister }) {
  const [mode, setMode] = useState("login"); // login | register
  const [placa, setPlaca] = useState("");
  const [pass, setPass] = useState("");
  const [err, setErr] = useState("");

```

```

const [ok, setOk] = useState("");
// Register fields
const [nombre, setNombre] = useState("");
const [rango, setRango] = useState("Policía");
const [adscripcion, setAdsc] = useState("");
const [pass2, setPass2] = useState("");

const handleLogin = () => {
  setErr("");
  if (!placa || !pass) { setErr("Completa todos los campos."); return; }
  const ok = onLogin({ placa, password: pass });
  if (!ok) setErr("Placa o contraseña incorrectos.");
};

const handleRegister = () => {
  setErr(""); setOk("");
  if (!placa || !nombre || !pass || !pass2) { setErr("Completa todos los campos."); return; }
  if (pass !== pass2) { setErr("Las contraseñas no coinciden."); return; }
  if (pass.length < 6) { setErr("La contraseña debe tener al menos 6 caracteres."); return; }
  const error = onRegister({ placa: placa.toUpperCase(), nombre, rango, adscripcion,
password: pass });
  if (error) { setErr(error); return; }
  setOk("¡Perfil creado! Ya puedes iniciar sesión.");
  setMode("login");
};

return (
  <div className="login-wrap">
    <div className="login-box">
      <div className="login-logo-wrap">
        <div className="login-logo">SSC</div>
        <div className="login-title">SSC CDMX</div>
        <div className="login-sub">Sistema Interno — Roleplay MX</div>
      </div>
      {err && <div className="error-msg">{Icon.warning} {err}</div>}
      {ok && <div className="success-msg">{ok}</div>}

      {mode === "login" ? (
        <>
          <div className="field">
            <label>Número de Placa</label>
            <input value={placa} onChange={e => setPlaca(e.target.value.toUpperCase())}
placeholder="Ej. MX-06" onKeyDown={e => e.key === "Enter" && handleLogin()} />

```

```
</div>
<div className="field">
  <label>Contraseña</label>
  <input type="password" value={pass} onChange={e => setPass(e.target.value)}
placeholder="••••••••" onKeyDown={e => e.key === "Enter" && handleLogin()} />
</div>
<button className="btn btn-primary" style={{ width: "100%", justifyContent: "center" }}
onClick={handleLogin}>
  Iniciar Sesión
</button>
<div className="divider" />
<div style={{ textAlign: "center", fontSize: 13, color: "var(--muted)" }}>
  ¿No tienes cuenta?{" "}
  <button className="link-btn" onClick={() => { setMode("register"); setErr(""); }}>
    Crear perfil
  </button>
</div>
</>
) : (
  <>
    <div className="field">
      <label>Número de Placa</label>
      <input value={placa} onChange={e => setPlaca(e.target.value.toUpperCase())}
placeholder="Ej. MX-07" />
    </div>
    <div className="field">
      <label>Nombre completo</label>
      <input value={nombre} onChange={e => setNombre(e.target.value)} placeholder="Tu
nombre en el RP" />
    </div>
    <div className="field">
      <label>Rango</label>
      <select value={rango} onChange={e => setRango(e.target.value)}>
        {[
          "Policía",
          "Cabo",
          "Sargento",
          "Suboficial",
          "Oficial",
          "Subinspector",
          "Inspector",
          "Subcomisario",
          "Comisario",
          "Director"
        ].map(r => <option key={r}>{r}</option>)}
      </select>
    </div>
    <div className="field">
      <label>Adscripción / Sector</label>
      <input value={adscripcion} onChange={e => setAdsc(e.target.value)} placeholder="Ej.
Sector Cuauhtémoc" />
    </div>
```



```

    <div className="field">
      <label>Contraseña</label>
      <input type="password" value={pass} onChange={e => setPass(e.target.value)}
placeholder="Mín. 6 caracteres" />
    </div>
    <div className="field">
      <label>Confirmar contraseña</label>
      <input type="password" value={pass2} onChange={e => setPass2(e.target.value)}
placeholder="Repite tu contraseña" />
    </div>
    <button className="btn btn-green" style={{ width: "100%", justifyContent: "center" }}
onClick={handleRegister}>
      {Icon.plus} Crear Perfil
    </button>
    <div className="divider" />
    <div style={{ textAlign: "center", fontSize: 13, color: "var(--muted)" }}>
      ¿Ya tienes cuenta?{" "}
      <button className="link-btn" onClick={() => { setMode("login"); setErr(""); }}>
        Iniciar sesión
      </button>
    </div>
  </>
)}
</div>
</div>
);
}

```

// — HOME

```

function HomeView({ session, onNav, elements, admins }) {
  const isSA = session.role === "superadmin";
  const isAdm = session.role === "admin" || isSA;

  const cards = [
    isSA && { id: "admins", icon: "★", title: "Gestión de Admins", desc: "Asigna y revoca permisos de administrador." },
    isAdm && { id: "elementos", icon: "🛡️", title: "Asuntos Internos", desc: "Expedientes, sanciones y registros del personal." },
    { id: "reportes", icon: "📋", title: "Reportes", desc: "Crea o consulta reportes de incidentes." },
    { id: "miperfil", icon: "👤", title: "Mi Perfil", desc: "Consulta tu expediente personal." },
  ]

```

```

].filter(Boolean);

return (
  <>
    <div style={{ marginBottom: 8 }}>
      <h2 style={{ fontSize: 26, color: "var(--white)" }}>Bienvenido, {session.nombre}</h2>
      <p style={{ color: "var(--muted)", fontSize: 14 }}>Sistema Interno SSC CDMX — Roleplay
MX</p>
    </div>

    {isSA && (
      <div className="stats-row" style={{ marginTop: 24 }}>
        <div className="stat-card">
          <div className="stat-value">{elements.length}</div>
          <div className="stat-label">Elementos Registrados</div>
        </div>
        <div className="stat-card">
          <div className="stat-value">{admins.length}</div>
          <div className="stat-label">Administradores</div>
        </div>
        <div className="stat-card">
          <div className="stat-value">{elements.filter(e => e.status === "activo").length}</div>
          <div className="stat-label">Activos</div>
        </div>
        <div className="stat-card">
          <div className="stat-value">{elements.reduce((a, e) => a + (e.records?.length || 0),
0)}</div>
          <div className="stat-label">Total Registros</div>
        </div>
      </div>
    )}

    <div className="cards-grid" style={{ marginTop: isSA ? 0 : 24 }}>
      {cards.map(c => (
        <div key={c.id} className="menu-card" onClick={() => onNav(c.id)}>
          <div className="menu-card-icon">{c.icon}</div>
          <div className="menu-card-title">{c.title}</div>
          <div className="menu-card-desc">{c.desc}</div>
        </div>
      ))}
    </div>
  </>
);

```

```
}
```

```
// ——— ADMINS VIEW (superadmin only)
```

```
function AdminsView({ admins, elements, onSave, toast }) {
  const [modal, setModal] = useState(false);
  const [placa, setPlaca] = useState("");
  const [nombre, setNombre] = useState("");
  const [pass, setPass] = useState("");
  const [err, setErr] = useState("");

  const add = () => {
    setErr("");
    const p = placa.trim().toUpperCase();
    if (!p || !nombre || !pass) { setErr("Completa todos los campos."); return; }
    if (admins.find(a => a.placa === p)) { setErr("Esa placa ya es admin."); return; }
    onSave([...admins, { placa: p, nombre, password: pass, addedAt: now() }]);
    toast("Admin agregado: " + p);
    setModal(false); setPlaca(""); setNombre(""); setPass("");
  };

  const remove = (placa) => {
    if (!confirm("¿Revocar acceso admin a " + placa + "?")) return;
    onSave(admins.filter(a => a.placa !== placa));
    toast("Admin revocado.");
  };

  return (
    <>
      <div className="section-hdr">
        <h2>{Icon.star} Administradores</h2>
        <button className="btn btn-gold" onClick={() => setModal(true)}>{Icon.plus} Agregar
Admin</button>
      </div>

      <div className="tbl-wrap">
        {admins.length === 0 ? (
          <div className="empty">No hay administradores aún.</div>
        ) : (
          <table>
            <thead><tr><th>Placa</th><th>Nombre</th><th>Agregado</th><th></th></tr></thead>
            <tbody>
              {admins.map(a => (
```

```

        <tr key={a.placa}>
          <td><span className="mono">{a.placa}</span></td>
          <td>{a.nombre}</td>
          <td style={{ color: "var(--muted)", fontSize: 12 }}>{a.addedAt}</td>
          <td>
            <button className="btn btn-red btn-sm" onClick={() =>
remove(a.placa)}>{Icon.trash} Revocar</button>
          </td>
        </tr>
      </tbody>
    </table>
  )}
</div>

{modal && (
  <Modal title={<>{Icon.star} Nuevo Administrador</>} onClose={() => setModal(false)}>
    {err && <div className="error-msg">{Icon.warning} {err}</div>}
    <div className="field"><label>Placa</label><input value={placa} onChange={e =>
setPlaca(e.target.value)} placeholder="MX-XX" /></div>
    <div className="field"><label>Nombre</label><input value={nombre} onChange={e =>
setNombre(e.target.value)} placeholder="Nombre en el RP" /></div>
    <div className="field"><label>Contraseña para este admin</label><input
type="password" value={pass} onChange={e => setPass(e.target.value)} /></div>
    <button className="btn btn-gold" style={{ marginTop: 8 }} onClick={add}>{Icon.check}
Confirmar</button>
  </Modal>
)}
</>
);
}

```

// — ELEMENTOS VIEW

```

function ElementosView({ session, elements, admins, onSave, toast }) {
  const isSA = session.role === "superadmin";
  const [search, setSearch] = useState("");
  const [selected, setSelected] = useState(null);
  const [modal, setModal] = useState(false);

  const filtered = elements.filter(e =>
    e.placa.includes(search.toUpperCase()) ||
    e.nombre.toLowerCase().includes(search.toLowerCase())
  )

```

```
);
```

```
const statusCls = { activo: "chip-active", baja: "chip-baja", suspendido: "chip-suspend" };  
const statusLabel = { activo: "Activo", baja: "Baja", suspendido: "Suspendido" };
```

```
if (selected) {  
  return (  
    <ExpedienteView  
      element={selected}  
      session={session}  
      onBack={() => setSelected(null)}  
      onSave={(updated) => {  
        const newEls = elements.map(e => e.placa === updated.placa ? updated : e);  
        onSave(newEls);  
        setSelected(updated);  
        toast("Expediente actualizado.");  
      }}  
      onDelete={() => {  
        if (!confirm("¿Eliminar elemento " + selected.placa + "?")) return;  
        onSave(elements.filter(e => e.placa !== selected.placa));  
        setSelected(null);  
        toast("Elemento eliminado.");  
      }}  
    />  
  );  
}
```

```
return (  
  <>  
    <div className="section-hdr">  
      <h2>{Icon.users} Personal SSC</h2>  
      {isSA && <button className="btn btn-primary" onClick={() => setModal(true)}>{Icon.plus}  
Crear Elemento</button>  
      </div>  
  
    <div className="field search-box" style={{ maxWidth: 320, marginBottom: 16 }}>  
      <input value={search} onChange={e => setSearch(e.target.value)} placeholder="Buscar  
por placa o nombre..." />  
    </div>  
  
    <div className="tbl-wrap">  
      {filtered.length === 0 ? (  
        <div className="empty">No se encontraron elementos.</div>  
      ) : (  
        <table>  
          <thead>  
            <tr>  
              <th>Placa</th>  
              <th>Nombre</th>  
              <th>Estado</th>  
            </tr>  
          </thead>  
          <tbody>  
            {filtered.map(e => (  
              <tr>  
                <td>{e.placa}</td>  
                <td>{e.nombre}</td>  
                <td>{e.estado}</td>  
              </tr>  
            ))}  
          </tbody>  
        </table>  
      )}  
    </div>  
  </>  
)
```

```

): (
  <table>
    <thead>

<tr><th>Placa</th><th>Nombre</th><th>Rango</th><th>Adscripción</th><th>Estado</th><th>
Registros</th><th></th></tr>
    </thead>
    <tbody>
      {filtered.map(e => (
        <tr key={e.placa}>
          <td><span className="mono">{e.placa}</span></td>
          <td style={{ color: "var(--white)", fontWeight: 500 }}>{e.nombre}</td>
          <td style={{ fontSize: 12, color: "var(--muted)" }}>{e.rango}</td>
          <td style={{ fontSize: 12 }}>{e.adscripcion || "—"}</td>
          <td><span className={`chip ${statusCls[e.status] || "chip-active"}`}>●
{statusLabel[e.status] || "Activo"}</span></td>
          <td style={{ fontSize: 12, color: "var(--muted)" }}>{e.records?.length || 0}</td>
          <td>
            <button className="btn btn-ghost btn-sm" onClick={() =>
setSelected(e)}>{Icon.file} Ver</button>
          </td>
        </tr>
      )})
    </tbody>
  </table>
)}
</div>

{modal && (
  <CrearElementoModal
    onClose={() => setModal(false)}
    onCreate={(data) => {
      if (elements.find(e => e.placa === data.placa)) { return "Placa ya registrada."; }
      onSave([...elements, { ...data, records: [], createdAt: now(), createdBy: session.placa,
status: "activo" }]);
      toast("Elemento creado: " + data.placa);
      setModal(false);
      return null;
    }}
  />
)}
</>
);

```

```
}
```

```
function CrearElementoModal({ onClose, onCreate }) {
  const [placa, setPlaca] = useState("");
  const [nombre, setNombre] = useState("");
  const [rango, setRango] = useState("Policía");
  const [adsc, setAdsc] = useState("");
  const [pass, setPass] = useState("");
  const [err, setErr] = useState("");

  const submit = () => {
    setErr("");
    if (!placa || !nombre || !pass) { setErr("Placa, nombre y contraseña son obligatorios."); return; }
    const e = onCreate({ placa: placa.toUpperCase(), nombre, rango, adscripcion: adsc,
password: pass });
    if (e) setErr(e);
  };

  return (
    <Modal title={<>{Icon.plus} Crear Elemento</>} onClose={onClose}>
      {err && <div className="error-msg">{Icon.warning} {err}</div>}
      <div className="field"><label>Placa</label><input value={placa} onChange={e =>
setPlaca(e.target.value)} placeholder="MX-XX" /></div>
      <div className="field"><label>Nombre</label><input value={nombre} onChange={e =>
setNombre(e.target.value)} /></div>
      <div className="field">
        <label>Rango</label>
        <select value={rango} onChange={e => setRango(e.target.value)}>

          {[
            "Policía", "Cabo", "Sargento", "Suboficial", "Oficial", "Subinspector", "Inspector", "Subcomisario", "Comisario", "Director"
          ].map(r => <option key={r}>{r}</option>)}

        </select>
      </div>
      <div className="field"><label>Adscripción / Sector</label><input value={adsc}
onChange={e => setAdsc(e.target.value)} /></div>
      <div className="field"><label>Contraseña inicial</label><input type="password"
value={pass} onChange={e => setPass(e.target.value)} /></div>
      <button className="btn btn-primary" style={{ marginTop: 8 }}
onClick={submit}>{Icon.check} Crear</button>
    </Modal>
  );
}
```

// — EXPEDIENTE

```
function ExpedienteView({ element, session, onBack, onSave, onDelete }) {
  const [tab, setTab] = useState("info");
  const [el, setEl] = useState(element);
  const isAdmin = session.role === "admin" || session.role === "superadmin";
  const [editInfo, setEditInfo] = useState(false);

  // Record modal
  const [recModal, setRecModal] = useState(false);
  const [recType, setRecType] = useState("sancion");
  const [recDesc, setRecDesc] = useState("");
  const [recAutor, setRecAutor] = useState(session.nombre + " (" + session.placa + ")");

  const addRecord = () => {
    if (!recDesc) return;
    const updated = { ...el, records: [...(el.records || []), { id: Date.now(), type: recType, desc:
recDesc, autor: recAutor, fecha: now() }] };
    setEl(updated); onSave(updated);
    setRecModal(false); setRecDesc("");
  };

  const delRecord = (id) => {
    if (!confirm("¿Eliminar este registro?")) return;
    const updated = { ...el, records: el.records.filter(r => r.id !== id) };
    setEl(updated); onSave(updated);
  };

  const updateStatus = (s) => {
    const updated = { ...el, status: s };
    setEl(updated); onSave(updated);
  };

  const tagCls = { sancion: "tag-sancion", elogio: "tag-elogio", nota: "tag-nota" };
  const recCls = { sancion: "sancion", elogio: "elogio", nota: "nota" };
  const statusCls = { activo: "chip-active", baja: "chip-baja", suspendido: "chip-suspend" };

  return (
    <>
      <div style={{ marginBottom: 4 }}>
        <button className="back-btn" onClick={onBack}>← Volver al listado</button>
      </div>
    </>
  );
}
```



```

<div className="profile-card" style={{ marginTop: 16 }}>
  <div className="profile-header">
    <div className="profile-avatar">{el.nombre?.[0] || "?"}</div>
    <div>
      <div className="profile-name">{el.nombre}</div>
      <div className="profile-placa">{el.placa}</div>
      <div className="profile-rank">{el.rango} — {el.adscripcion || "Sin adscripción"}</div>
      <div style={{ marginTop: 8 }}>
        <span className={`chip ${statusCls[el.status] || "chip-active"}`>● {el.status ||
"Activo"}</span>
      </div>
    </div>
    {isAdmin && (
      <div style={{ marginLeft: "auto", display: "flex", gap: 8, flexWrap: "wrap", alignItems:
"flex-start" }}>
        <select
          value={el.status || "activo"}
          onChange={e => updateStatus(e.target.value)}
          style={{ padding: "6px 10px", borderRadius: 5, background: "var(--card)", border: "1px
solid var(--border)", color: "var(--text)", fontSize: 12, fontFamily: "Barlow, sans-serif" }}
          >
          <option value="activo">Activo</option>
          <option value="suspendido">Suspendido</option>
          <option value="baja">Baja</option>
        </select>
        <button className="btn btn-red btn-sm" onClick={onDelete}>{Icon.trash}
Eliminar</button>
      </div>
    )}
  </div>

  <div className="tabs">
    {["info", "registros"].map(t => (
      <button key={t} className={`tab-btn ${tab === t ? "active" : ""}`} onClick={() =>
setTab(t)}>
        {t === "info" ? "📄 Información" : "📁 Registros (${el.records?.length || 0})`}
      </button>
    ))}
  </div>

  {tab === "info" && (
    <EditableInfo el={el} isAdmin={isAdmin} onSave={(updated) => { setEl(updated);

```

```

onSave(updated); }} />
  })

  {tab === "registros" && (
    <>
      {isAdmin && (
        <div style={{ marginBottom: 14 }}>
          <button className="btn btn-primary btn-sm" onClick={() =>
setRecModal(true)}>{Icon.plus} Agregar Registro</button>
        </div>
      )}
      {(!el.records || el.records.length === 0) ? (
        <div className="empty">Sin registros en el expediente.</div>
      ) : (
        <div className="record-list">
          {[...el.records].reverse().map(r => (
            <div key={r.id} className={`record-item ${recCls[r.type]}`>
              <div style={{ flex: 1 }}>
                <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom: 5 }}>
                  <span className={`tag ${tagCls[r.type]}`}>{r.type}</span>
                </div>
                <div style={{ fontSize: 14, color: "var(--white)" }}>{r.desc}</div>
                <div className="record-meta">Por: {r.autor} — {r.fecha}</div>
              </div>
              {isAdmin && (
                <button className="btn btn-red btn-sm" style={{ flexShrink: 0 }} onClick={() =>
delRecord(r.id)}>{Icon.trash}</button>
              )}
            </div>
          )]}
        </div>
      )}
    </>
  )}
</div>

{recModal && (
  <Modal title={<>{Icon.file} Nuevo Registro</>} onClose={() => setRecModal(false)}>
    <div className="field">
      <label>Tipo</label>
      <select value={recType} onChange={e => setRecType(e.target.value)}>
        <option value="sancion">Sanción</option>
        <option value="elogio">Elogio</option>
      </select>
    </div>
  </Modal>
)

```

```

      <option value="nota">Nota</option>
    </select>
  </div>
  <div className="field">
    <label>Descripción</label>
    <textarea value={recDesc} onChange={e => setRecDesc(e.target.value)}
placeholder="Detalla el registro..." />
  </div>
  <div className="field">
    <label>Registrado por</label>
    <input value={recAutor} onChange={e => setRecAutor(e.target.value)} />
  </div>
  <button className="btn btn-primary" style={{ marginTop: 8 }}
onClick={addRecord}>{Icon.check} Guardar</button>
</Modal>
  )}
</>
);
}

```

```

function EditableInfo({ el, isAdmin, onSave }) {
  const [editing, setEditing] = useState(false);
  const [d, setD] = useState({ ...el });

  const save = () => { onSave(d); setEditing(false); };

  const field = (label, key, type = "text", opts = null) => (
    <div className="info-item">
      <label>{label}</label>
      {editing ? (
        opts
        ? <select value={d[key] || ""} onChange={e => setD({ ...d, [key]: e.target.value })} style={{
padding: "8px 10px", borderRadius: 5, background: "var(--card)", border: "1px solid
var(--border)", color: "var(--white)", fontSize: 13, fontFamily: "Barlow, sans-serif", width: "100%"
}}>
          {opts.map(o => <option key={o}>{o}</option>)}
        </select>
        : <input value={d[key] || ""} type={type} onChange={e => setD({ ...d, [key]: e.target.value
})} style={{ padding: "8px 10px", borderRadius: 5, background: "var(--card)", border: "1px solid
var(--border)", color: "var(--white)", fontSize: 13, fontFamily: "Barlow, sans-serif", width: "100%",
outline: "none" }} />
      ) : (
        <span>{el[key] || "—"}</span>

```

```

    })
  </div>
);

return (
  <>
    {isAdmin && (
      <div style={{ marginBottom: 14 }}>
        {editing
          ? <><button className="btn btn-green btn-sm" onClick={save}>{Icon.check}
Guardar</button> <button className="btn btn-ghost btn-sm" style={{ marginLeft: 6 }}
onClick={() => setEditing(false)}>Cancelar</button></>
          : <button className="btn btn-ghost btn-sm" onClick={() => setEditing(true)}>{Icon.edit}
Editar Info</button>
        }
      </div>
    )}
    <div className="info-grid">
      {field("Nombre", "nombre")}
      {field("Placa", "placa")}
      {field("Rango", "rango", "text",
["Policía", "Cabo", "Sargento", "Suboficial", "Oficial", "Subinspector", "Inspector", "Subcomisario", "Co
misario", "Director"])}
      {field("Adscripción", "adscripcion")}
      {field("Turno", "turno", "text", ["Matutino", "Vespertino", "Nocturno", "Mixto"])}
      {field("CURP / ID", "curp")}
      <div className="info-item">
        <label>Registrado</label>
        <span style={{ fontSize: 12, color: "var(--muted)" }}>{el.createdAt || "—"}</span>
      </div>
      <div className="info-item">
        <label>Creado por</label>
        <span style={{ fontSize: 12, color: "var(--muted)" }}>{el.createdBy || "—"}</span>
      </div>
    </div>
    {editing && (
      <div style={{ marginTop: 14 }}>
        <div className="field">
          <label>Notas generales</label>
          <textarea value={d.notas || ""} onChange={e => setD({ ...d, notas: e.target.value })} />
        </div>
      </div>
    )}
  </>
)

```

```

    {!editing && el.notas && (
      <div style={{ marginTop: 14 }}>
        <div style={{ fontSize: 11, fontWeight: 600, letterSpacing: ".1em", textTransform:
"uppercase", color: "var(--muted)", marginBottom: 6 }}>Notas generales</div>
        <p style={{ fontSize: 13, color: "var(--text)", lineHeight: 1.5 }}>{el.notas}</p>
      </div>
    )}
  </>
);
}

```

// — MI PERFIL

```

function MiPerfilView({ session, elements, onSave, toast }) {
  const el = elements.find(e => e.placa === session.placa);
  const isSA = session.placa === SUPER_ADMIN_PLACA;

  if (isSA) {
    return (
      <div className="profile-card">
        <div className="profile-header">
          <div className="profile-avatar">K</div>
          <div>
            <div className="profile-name">kracccea</div>
            <div className="profile-placa">{SUPER_ADMIN_PLACA}</div>
            <div className="profile-rank">Director General de Asuntos Internos</div>
          </div>
        </div>
        <div style={{ color: "var(--muted)", fontSize: 14 }}>Cuenta de Super Administrador —
acceso total al sistema.</div>
      </div>
    );
  }

  if (!el) {
    return (
      <div className="profile-card">
        <div className="empty">Tu perfil aún no está en el sistema. Espera a que un
administrador te agregue o verifica tu registro.</div>
      </div>
    );
  }
}

```

```

const statusCls = { activo: "chip-active", baja: "chip-baja", suspendido: "chip-suspend" };
const tagCls = { sancion: "tag-sancion", elogio: "tag-elogio", nota: "tag-nota" };
const recCls = { sancion: "sancion", elogio: "elogio", nota: "nota" };

return (
  <div className="profile-card">
    <div className="profile-header">
      <div className="profile-avatar">{el.nombre?.[0]}</div>
      <div>
        <div className="profile-name">{el.nombre}</div>
        <div className="profile-placa">{el.placa}</div>
        <div className="profile-rank">{el.rango} — {el.adscripcion || "Sin adscripción"}</div>
        <div style={{ marginTop: 8 }}>
          <span className={`chip ${statusCls[el.status] || "chip-active"}`}>● {el.status ||
"Activo"}</span>
        </div>
      </div>
    </div>

    <div className="info-grid" style={{ marginBottom: 20 }}>
      <div className="info-item"><label>Rango</label><span>{el.rango}</span></div>
      <div className="info-item"><label>Adscripción</label><span>{el.adscripcion ||
"—"}</span></div>
      <div className="info-item"><label>Turno</label><span>{el.turno || "—"}</span></div>
      <div className="info-item"><label>CURP / ID</label><span>{el.curp ||
"—"}</span></div>
    </div>

    <div style={{ fontSize: 14, fontWeight: 600, color: "var(--muted)", marginBottom: 10,
textTransform: "uppercase", letterSpacing: ".08em" }}>
      Mi Expediente ({el.records?.length || 0} registros)
    </div>

    {(!el.records || el.records.length === 0) ? (
      <div className="empty">Sin registros en tu expediente.</div>
    ) : (
      <div className="record-list">
        {[...el.records].reverse().map(r => (
          <div key={r.id} className={`record-item ${recCls[r.type]}`}>
            <div>
              <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom: 5 }}>
                <span className={`tag ${tagCls[r.type]}`}>{r.type}</span>

```

```

        </div>
        <div style={{ fontSize: 14, color: "var(--white)" }}>{r.desc}</div>
        <div className="record-meta">Por: {r.autor} — {r.fecha}</div>
    </div>
</div>
    )}
</div>
    }
</div>
);
}

```

// ——— REPORTES

```

function ReportesView({ session, elements, reports, onSave, toast }) {
    const [modal, setModal] = useState(false);
    const [tipo, setTipo] = useState("Incidente");
    const [desc, setDesc] = useState("");
    const [involucrados, setInv] = useState("");

    const myReports = session.role === "superadmin" || session.role === "admin"
        ? reports
        : reports.filter(r => r.autorPlaca === session.placa);

    const create = () => {
        if (!desc) return;
        const r = { id: Date.now(), tipo, desc, involucrados, autorPlaca: session.placa, autorNombre:
session.nombre, fecha: now() };
        onSave([r, ...reports]);
        toast("Reporte creado.");
        setModal(false); setDesc(""); setInv("");
    };

    const del = (id) => {
        if (!confirm("¿Eliminar reporte?")) return;
        onSave(reports.filter(r => r.id !== id));
        toast("Reporte eliminado.");
    };

    const canDelete = (r) => session.role === "superadmin" || session.role === "admin" ||
r.autorPlaca === session.placa;

```

```

return (
  <>
    <div className="section-hdr">
      <h2>{Icon.file} Reportes</h2>
      <button className="btn btn-primary" onClick={() => setModal(true)}>{Icon.plus} Nuevo
Reporte</button>
    </div>

    {myReports.length === 0 ? (
      <div className="tbl-wrap"><div className="empty">No hay reportes aún.</div></div>
    ) : (
      <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
        {myReports.map(r => (
          <div key={r.id} className="record-item nota" style={{ background: "var(--card)", border:
"1px solid var(--border)", borderLeft: "3px solid var(--accent)" }}>
            <div style={{ flex: 1 }}>
              <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom: 6 }}>
                <span className="tag tag-nota">{r.tipo}</span>
                <span style={{ fontSize: 12, color: "var(--muted)" }}>{r.fecha}</span>
              </div>
              <p style={{ fontSize: 14, color: "var(--white)", lineHeight: 1.5 }}>{r.desc}</p>
              {r.involucrados && <div className="record-meta" style={{ marginTop: 6
}}>Involucrados: {r.involucrados}</div>}
              <div className="record-meta">Por: {r.autorNombre} ({r.autorPlaca})</div>
            </div>
            {canDelete(r) && (
              <button className="btn btn-red btn-sm" style={{ flexShrink: 0 }} onClick={() =>
del(r.id)}>{Icon.trash}</button>
            )}
          </div>
        ))}
      </div>
    )}

    {modal && (
      <Modal title={<>{Icon.file} Nuevo Reporte</>} onClose={() => setModal(false)}>
        <div className="field">
          <label>Tipo de reporte</label>
          <select value={tipo} onChange={e => setTipo(e.target.value)}>
            {["Incidente", "Informe de patrullaje", "Queja ciudadana", "Uso de
fuerza", "Accidente", "Otro"].map(t => <option key={t}>{t}</option>)}
          </select>
        </div>
      </Modal>
    )}
  </>
)

```



```

    <div className="field">
      <label>Descripción</label>
      <textarea value={desc} onChange={e => setDesc(e.target.value)} placeholder="Detalla
el reporte..." style={{ minHeight: 100 }} />
    </div>
    <div className="field">
      <label>Elementos involucrados (opcional)</label>
      <input value={involucrados} onChange={e => setInv(e.target.value)}
placeholder="Placas o nombres" />
    </div>
    <button className="btn btn-primary" style={{ marginTop: 8 }}
onClick={create}>{Icon.check} Crear Reporte</button>
  </Modal>
)}
</>
);
}

```